C PROGRAMMING – ASSIGNMENT 1

Course Code:	CSA0222	**Course Title:**	C PROGRAMMING
CO Assessed:	CO2	**Total Marks:**	50
CO2 Statement:	Examine and write solutions for problems using operators, decision-making and looping constructs – arithmetic, relational, logical, bitwise, assignment and conditional operators, precedence and associativity, if, if–else, else–if ladder, nested if, switch, while, do–while, for, break, continue and goto.		
Student Name:	Vishnu Raja V	**Reg. No.:**	1921260143

INTRODUCTION

C programming provides different operators and control statements to perform calculations, make decisions and repeat instructions. Operators such as arithmetic, relational, logical, bitwise and conditional operators help solve different programming problems.

Decision-making statements such as if, if-else, else-if and switch allow a program to select actions based on conditions. Looping statements such as for, while and do-while are used to repeat operations until a particular condition is satisfied.

This assignment examines the use of these concepts through practical problems. Each question includes problem analysis, program implementation, predicted results, observed outputs, testing and complexity analysis. The final question focuses on identifying a limitation in an existing solution and improving it through redesign.

Q1 – PRECEDENCE, ASSOCIATIVITY AND THE DISCOUNT BUG

1.1 Problem Analysis

A government pharmacy calculates the amount payable by a customer based on the product price and quantity. Senior citizens receive a 15% discount, and 5% GST is added to the bill.

The original developer used a single expression without intermediate variables. This can make the calculation difficult to understand and may produce unexpected results because of operator precedence and integer division.

The original expressions are:

```
total = price * qty - price * qty * 15 / 100 * senior
```

```
        + price * qty * 5 / 100;

net = total > 1000 ? total - 50 : total;

count = count++ + 1;
```

1.2 Test Data and Predicted Results

The following three test cases are selected to examine normal billing, senior-citizen discount and integer division.

Test Case	Price	Quantity	Senior Citizen
1	100	2	0
2	200	5	1
3	99	1	1

The original calculation uses integer arithmetic.

Test Case	Gross Amount	Discount	GST	Total	Net Amount
1	200	0	10	210	210
2	1000	150	50	900	900
3	99	14	4	89	89

1.3 Operator Precedence and Associativity

In C, multiplication, division and modulus have higher precedence than addition and subtraction. Operators with the same precedence are generally evaluated from left to right for these arithmetic operations.

The original expression is evaluated by performing multiplication and division before addition and subtraction.

The conditional operator `?:` checks whether the total is greater than 1000. If the condition is true, ₹50 is deducted; otherwise, the original total is retained.

The expression `count = count++ + 1` has undefined behaviour because `count` is modified more than once without the required sequencing.

1.4 Problems Identified

Expression or Concept	Problem
Long billing expression	Difficult to read and maintain
Integer division	Decimal portions are discarded
Discount calculation	Requires clear separation of gross amount and discount
GST calculation	Should be calculated on the correct taxable amount

`count = count++ + 1`	Causes undefined behaviour
`-Wall` compiler option	May warn about sequence-point or unsequenced-operation issues, depending on the compiler

1.5 Corrected Program

The calculation can be improved by using intermediate variables. This makes the billing process easier to understand and reduces calculation mistakes.

```
#include <stdio.h>

int main()
{
    double price, qty, gross, discount, taxable, gst, total, net;
    int senior;

    printf("Enter price: ");
    scanf("%lf", &price);

    printf("Enter quantity: ");
    scanf("%lf", &qty);

    printf("Senior citizen? (1 for Yes, 0 for No): ");
    scanf("%d", &senior);

    gross = price * qty;

    if (senior == 1)
        discount = gross * 15.0 / 100.0;
    else
        discount = 0.0;

    taxable = gross - discount;
    gst = taxable * 5.0 / 100.0;

    total = taxable + gst;

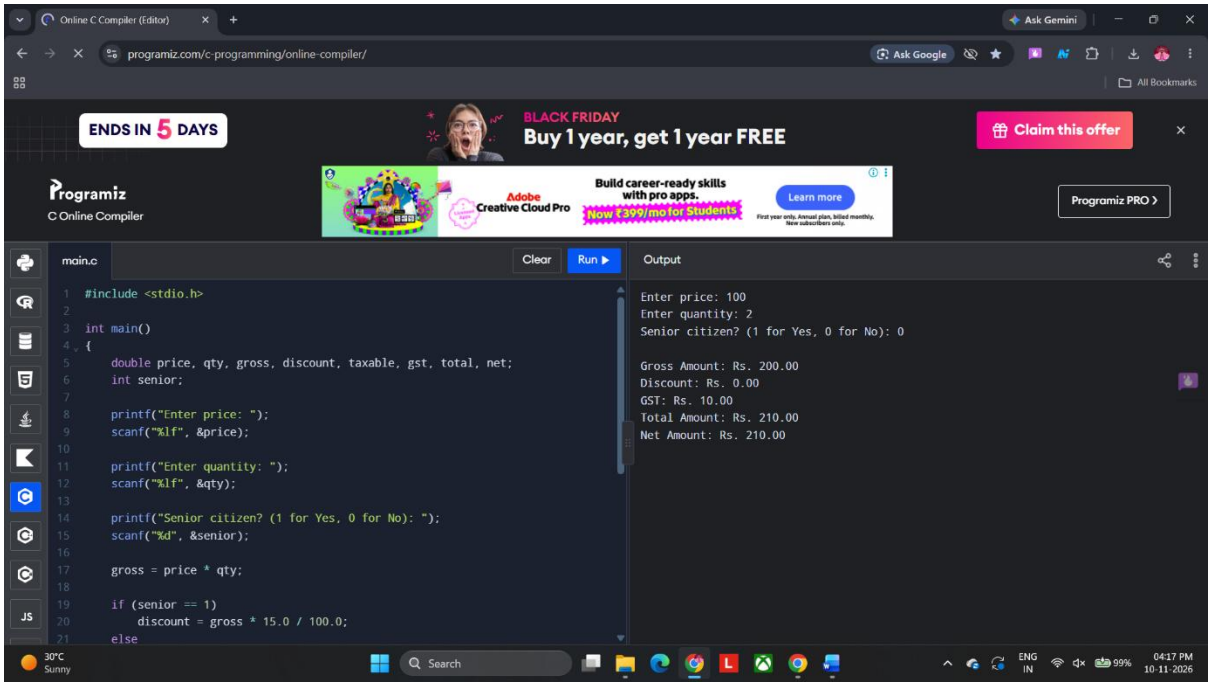
    net = (total > 1000.0) ? total - 50.0 : total;

    printf("\nGross Amount: Rs. %.2f", gross);
    printf("\nDiscount: Rs. %.2f", discount);
    printf("\nGST: Rs. %.2f", gst);
    printf("\nTotal Amount: Rs. %.2f", total);
    printf("\nNet Amount: Rs. %.2f\n", net);

    return 0;
}
```

1.6 Corrected Calculation Results

Test Case	Gross	Discount	Taxable Amount	GST	Net Amount
1	200.00	0.00	200.00	10.00	210.00
2	1000.00	150.00	850.00	42.50	892.50
3	99.00	14.85	84.15	4.21	88.36



The coupon deduction is applied only when the calculated total is greater than ₹1000. None of these three corrected test cases crosses that limit.

1.7 Alternative Concept and Complexity Analysis

Instead of writing the entire calculation in one expression, intermediate variables such as gross, discount, taxable amount and GST can be used.

Factor	Single Expression	Intermediate Variables
Time Complexity	O(1)	O(1)
Extra Space	O(1)	O(1)
Readability	Difficult	Easy
Debugging	Difficult	Easier
Accuracy	More chances of mistakes	Easier to verify

Conclusion: Using intermediate variables and appropriate brackets is better because it improves readability, testing and correctness without changing the overall time complexity.

Q2 – WHEN DOES SHORT-CIRCUIT EVALUATION MATTER?

2.1 Problem Analysis

A railway ticket counter grants a concession when a passenger is a senior citizen and the ticket class is eligible, or when the passenger holds a valid disability card.

The disability-card verification is assumed to be a slow function because it checks information against a server.

The condition is:

```
if ((senior && eligible) || verify_card(disability))
```

In C, the logical AND (`&&`) and logical OR (`||`) operators use short-circuit evaluation.

- In `A && B`, B is not evaluated when A is false.
- In `A || B`, B is not evaluated when A is true.

2.2 Designed Experiment

For this experiment, a sample dataset of 1,000 passengers is assumed.

Passenger Group	Number of Passengers
Senior citizens with eligible tickets	120
Other passengers	880
Total passengers	1000
Disability-card holders among the other passengers	150

These are sample values created for demonstrating short-circuit evaluation.

2.3 Comparison of Condition Orders

Order 1: Senior-citizen condition first

```
if ((senior && eligible) || verify_card(disability))
```

The verification function is skipped for the 120 passengers whose first condition is already true.

Number of verification calls:

1000 − 120 = 880 calls.

Order 2: Verification function first

```
if (verify_card(disability) || (senior && eligible))
```

The verification function is called for every passenger because it is the first condition.

Number of verification calls: 1000.

Condition Order	Verification Calls
Senior condition first	880
Verification first	1000
Difference	120

2.4 Difference Between Logical and Bitwise Operators

Logical Operator	Bitwise Operator			
&&	&			
`		`	`	`
Used mainly with conditions	Used mainly with individual bits			
Supports short-circuit evaluation	Evaluates both operands			
Returns 0 or 1 for logical results	Produces a bitwise result			

Example:

```
if (0 && verify_card(1))
    printf("Approved");
```

The function is not called because the first operand is false.

```
if (0 & verify_card(1))
    printf("Approved");
```

The function is evaluated even though the first operand is zero.

2.5 Sample C Program

```
#include <stdio.h>

int calls = 0;

int verify_card(int disability)
{
    calls++;
    return disability;
}

int main()
{
    int senior, eligible, disability;
    int approved = 0;

    for (int i = 0; i < 1000; i++)
    {
        senior = (i < 120);
        eligible = 1;
        disability = (i >= 120 && i < 270);

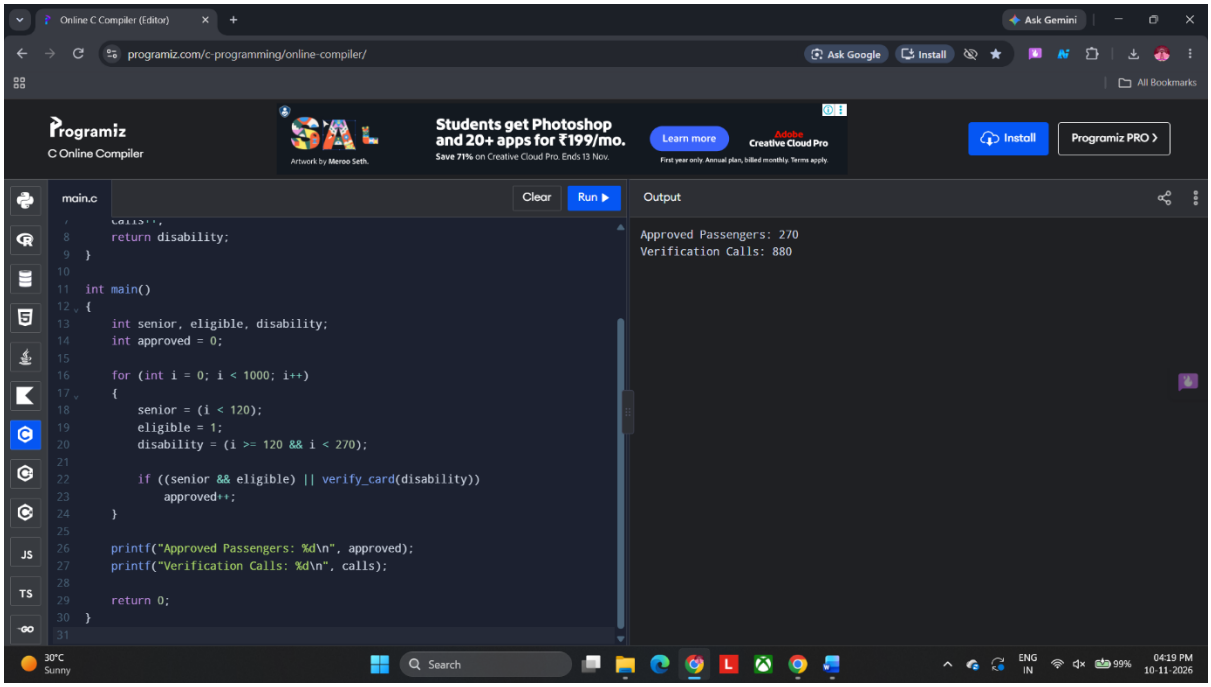
        if ((senior && eligible) || verify_card(disability))
            approved++;
    }

    printf("Approved Passengers: %d\n", approved);
    printf("Verification Calls: %d\n", calls);

    return 0;
}
```

Expected output:

```
Approved Passengers: 270
Verification Calls: 880
```



2.6 Changed Condition – Audit Logging

If the verification function must record every passenger in an audit log, it must be called for every passenger. In that case, skipping the function is not acceptable.

The verification should be performed separately:

```
int verified = verify_card(disability);

if ((senior && eligible) || verified)
{
    approved++;
}
```

This ensures that the verification function is executed for every passenger.

2.7 Alternative Concept and Complexity Analysis

Nested if statements can be used instead of a combined logical expression.

Method	Verification Calls in the Sample	Time Complexity
Short-circuit condition, senior first	880	O(n)
Verification first	1000	O(n)
Nested if with same short-circuit logic	880	O(n)

Bitwise operators with verification	1000	O(n)

Conclusion: The cheapest or most likely false condition should generally be placed first when it can prevent an expensive operation. However, if every verification must be recorded for auditing, the verification function must be called for every passenger.

Q3 – ENGINEER A DEVICE-STATUS REGISTER WITH BITWISE OPERATORS

3.1 Problem Analysis

A smart-home controller needs to store the ON/OFF status of up to eight devices. Instead of using a separate variable for every device, a single unsigned char can store all eight states.

Each bit represents one device.

Device	Bit Position
LIGHT	0
FAN	1
AC	2
HEATER	3
PUMP	4
DEVICE 6	5
DEVICE 7	6
DEVICE 8	7

3.2 Bitwise Operations

Operation	Expression	Purpose
Switch ON	`reg | (1u << dev)`	Sets a bit to 1
Switch OFF	`reg & ~(1u << dev)`	Clears a bit
Toggle	`reg ^ (1u << dev)`	Flips a bit
Check ON	`(reg & (1u << dev)) != 0`	Checks the bit

3.3 Predicted Register Values

The operations in the given program are:

1. Switch ON LIGHT.
2. Switch ON AC.
3. Display the register.
4. Toggle FAN.

5. Switch OFF LIGHT.
6. Display the register.

Operation	Register in Binary	Decimal Value
Initial register	00000000	0
LIGHT ON	00000001	1
AC ON	00000101	5
FAN toggle	00000111	7
LIGHT OFF	00000110	6

3.4 Completed C Program

```
#include <stdio.h>

enum { LIGHT = 0, FAN = 1, AC = 2, HEATER = 3, PUMP = 4 };

unsigned char switch_on(unsigned char reg, int dev)
{
    return reg | (1u << dev);
}

unsigned char switch_off(unsigned char reg, int dev)
{
    return reg & ~(1u << dev);
}

unsigned char toggle(unsigned char reg, int dev)
{
    return reg ^ (1u << dev);
}

int is_on(unsigned char reg, int dev)
{
    return (reg & (1u << dev)) != 0;
}

void show(unsigned char reg)
{
    for (int bit = 7; bit >= 0; bit--)
        putchar(((reg >> bit) & 1u) ? '1' : '0');

    putchar('\n');
}

int main(void)
{
    unsigned char reg = 0;

    reg = switch_on(reg, LIGHT);
    reg = switch_on(reg, AC);
    show(reg);

    reg = toggle(reg, FAN);
    reg = switch_off(reg, LIGHT);
    show(reg);

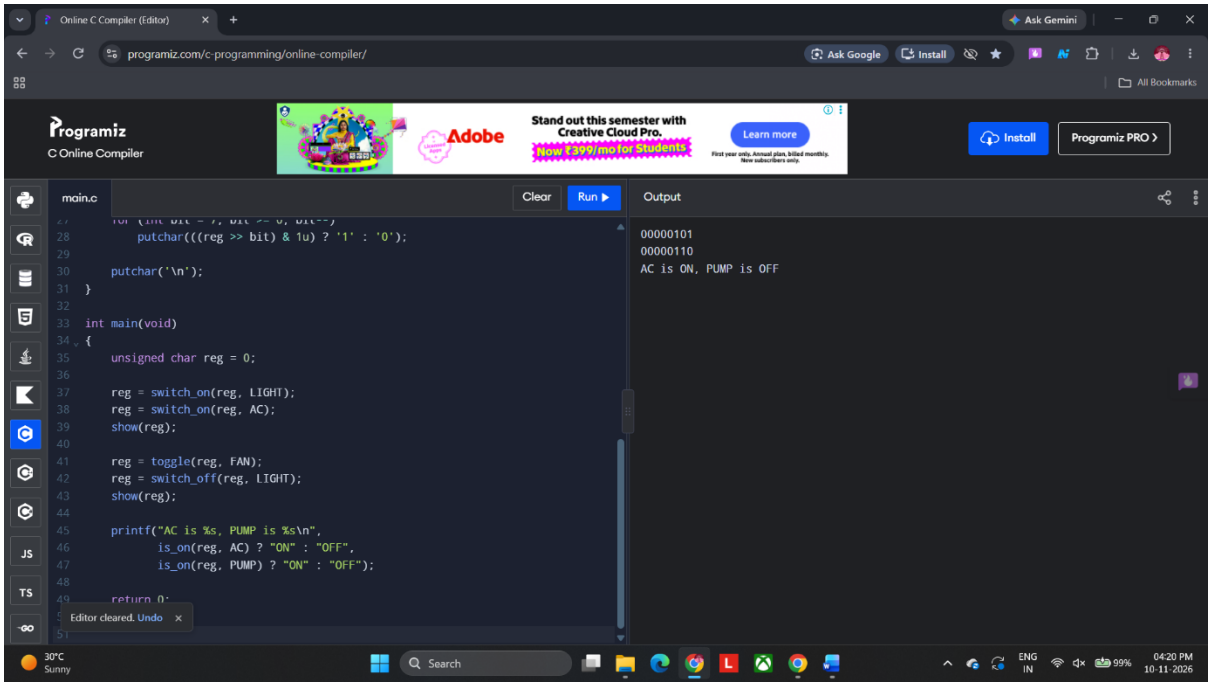
    printf("AC is %s, PUMP is %s\n",
           is_on(reg, AC) ? "ON" : "OFF",
```

```
            is_on(reg, PUMP) ? "ON" : "OFF");

    return 0;
}
```

3.5 Expected Output

```
00000101
00000110
AC is ON, PUMP is OFF
```



3.6 Explanation of Operator Precedence and Signedness

The expressions `reg & 1 << dev` and `(reg & 1) << dev` are different because the shift operator has higher precedence than the bitwise AND operator.

- `reg & 1 << dev` is evaluated as `reg & (1 << dev)`.
- `(reg & 1) << dev` first checks the lowest bit and then shifts the result.

An unsigned char is preferred because it represents the register as a collection of bits without signed-value interpretation. Using a signed char when bit 7 is set can cause implementation-dependent or confusing results when the value is promoted to int.

3.7 Alternative Concept – Array Storage

An alternative is to store the device states in an array:

```
char state[8] = {0};
```

Each element represents one device. A value of 1 means ON and 0 means OFF.

Factor	Bitwise Register	Character Array
Storage per home	1 byte	8 bytes
Switch ON/OFF	O(1)	O(1)
Toggle	O(1)	O(1)
Check status	O(1)	O(1)
Count ON devices	O(8)	O(8)

For 1,000,000 homes:

Storage Method	Total Storage
One-byte register	1,000,000 bytes
Eight-byte array	8,000,000 bytes

Conclusion: Bitwise storage is more memory-efficient when only ON/OFF information is required. Both methods provide constant-time operations for an individual device, but the bitwise method uses less memory.

Q4 – ELSE-IF LADDER OR SWITCH?

4.1 Problem Analysis

A university examination cell uses students' marks to assign grades.

The grade rules are:

Marks Range	Grade
90–100	S
80–89	A
70–79	B
60–69	C
50–59	D
Below 50	F
Outside 0–100	Invalid (?)

The program must reject marks below 0 or above 100.

4.2 Implementation Using Else-if Ladder

```
#include <stdio.h>

char grade_if(int marks)
{
    if (marks < 0 || marks > 100)
        return '?';
    else if (marks >= 90)
        return 'S';
    else if (marks >= 80)
```

```
        return 'A';
    else if (marks >= 70)
        return 'B';
    else if (marks >= 60)
        return 'C';
    else if (marks >= 50)
        return 'D';
    else
        return 'F';
}

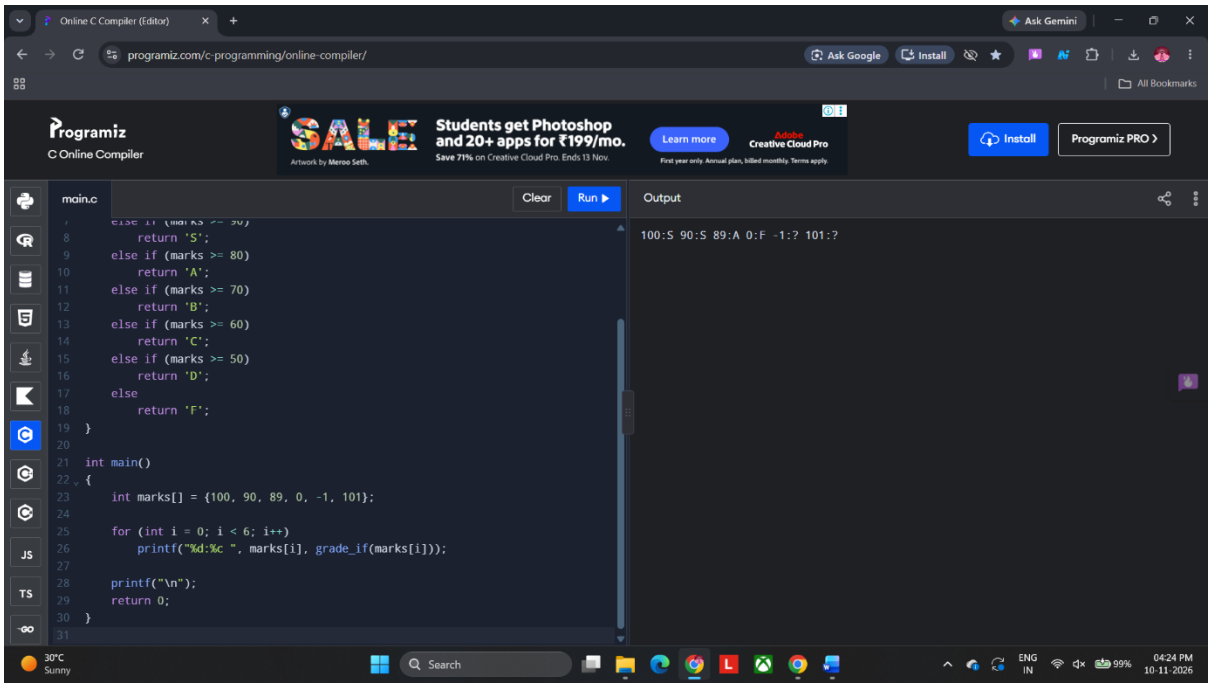
int main()
{
    int marks[] = {100, 90, 89, 0, -1, 101};

    for (int i = 0; i < 6; i++)
        printf("%d:%c ", marks[i], grade_if(marks[i]));

    printf("\n");
    return 0;
}
```

4.3 Expected Output of the Given Else-if Program

```
100:S 90:S 89:A 0:F -1:? 101:?
```



4.4 Implementation Using Switch

```
#include <stdio.h>

char grade(int marks)
{
    if (marks < 0 || marks > 100)
        return '?';
```

```
    switch (marks / 10)
    {
        case 10:
        case 9:
            return 'S';

        case 8:
            return 'A';

        case 7:
            return 'B';

        case 6:
            return 'C';

        case 5:
            return 'D';

        default:
            return 'F';
    }
}

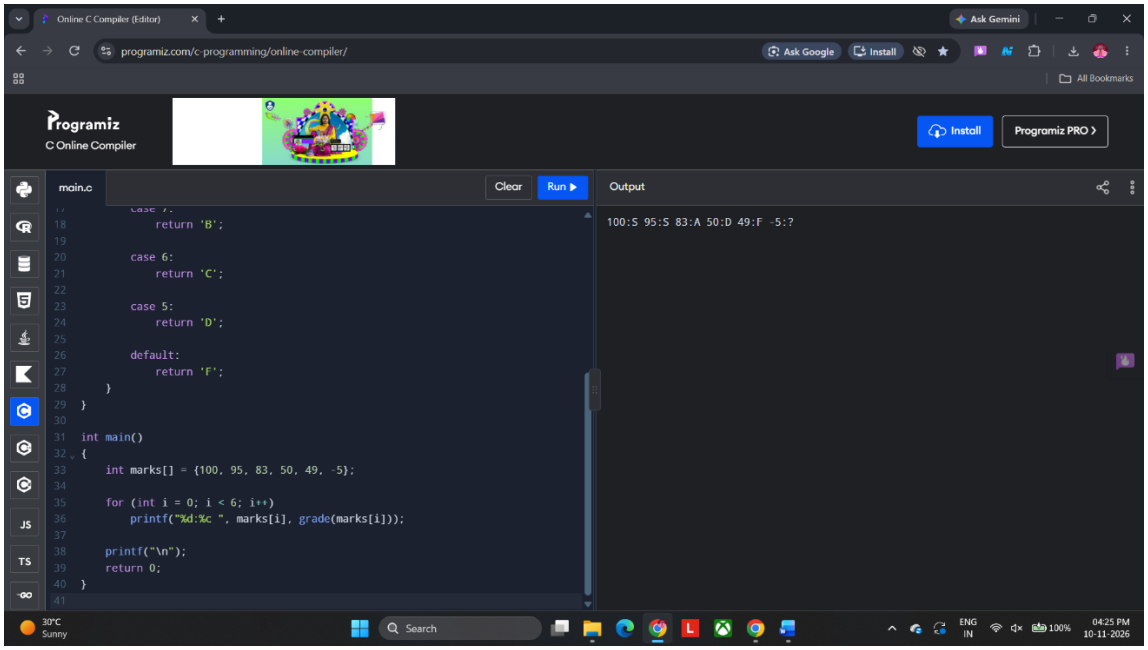
int main()
{
    int marks[] = {100, 95, 83, 50, 49, -5};

    for (int i = 0; i < 6; i++)
        printf("%d:%c ", marks[i], grade(marks[i]));

    printf("\n");
    return 0;
}
```

4.5 Expected Output of the Given Switch Program

```
100:S 95:S 83:A 50:D 49:F -5:?
```



4.6 Boundary Testing

The following values test the grade boundaries and invalid inputs.

Marks	Expected Grade
0	F
49	F
50	D
59	D
60	C
69	C
70	B
79	B
80	A
89	A
90	S
100	S
-1	?
101	?

4.7 Effect of a Missing Break

In a switch statement, `break` prevents execution from continuing into the next case. If a break is missing, the program may execute the statements in the following cases. This is called fall-through.

In the above implementation, each case uses `return`, so a break is not necessary. If the cases used assignments and print statements instead, break statements would be required to prevent fall-through.

4.8 Why Switch Cannot Directly Test Ranges

A switch statement compares one expression with constant case values. It cannot directly test a range such as 75–84.

An else-if ladder is more suitable for range-based conditions because it can use relational operators such as >= and <=.

4.9 Scholarship Decision Structure

The scholarship decision depends on both the student's grade and family income. A nested if statement can first identify the grade and then check the income category.

A lookup table can also be used for grade classification, but income-based scholarship rules may still require additional conditions.

4.10 Alternative Concept – Lookup Table

```
char grades[11] = {
    'F', 'F', 'F', 'F', 'F',
    'D', 'C', 'B', 'A', 'S', 'S'
};

char grade_lookup(int marks)
{
    if (marks < 0 || marks > 100)
        return '?';

    return grades[marks / 10];
}
```

Method	Time Complexity	Extra Space
Else-if ladder	O(k) worst case	O(1)
Switch	O(1) for this fixed classification	O(1)
Lookup table	O(1)	O(1) fixed table

Conclusion: The else-if ladder is easier to modify when grade boundaries change. The switch statement and lookup table provide simple classification when the ranges can be mapped safely to integer categories.

Q5 – COUNT THE WORK DONE BY A LOOP

5.1 Problem Analysis

A college fest awards prizes to tickets whose numbers are prime. A prime number is a number greater than 1 that has exactly two factors: 1 and itself.

The basic method checks every divisor from 2 to n − 1. This performs unnecessary operations for large numbers.

The improved method uses the mathematical principle that if a number has a divisor greater than its square root, it must also have a divisor smaller than its square root.

Therefore, checking divisors only up to the square root is sufficient.

To avoid calling `sqrt()`, the loop condition can be written as:

```
i <= n / i
```

This also avoids possible overflow from `i * i`.

5.2 Basic Prime-Checking Program

```
#include <stdio.h>

int is_prime_basic(unsigned long n, unsigned long *checks)
```

```
{
    *checks = 0;

    if (n < 2)
        return 0;

    for (unsigned long i = 2; i < n; i++)
    {
        (*checks)++;

        if (n % i == 0)
            return 0;
    }

    return 1;
}

int main()
{
    unsigned long numbers[] = {97, 100, 7919, 1000003};
    unsigned long checks;

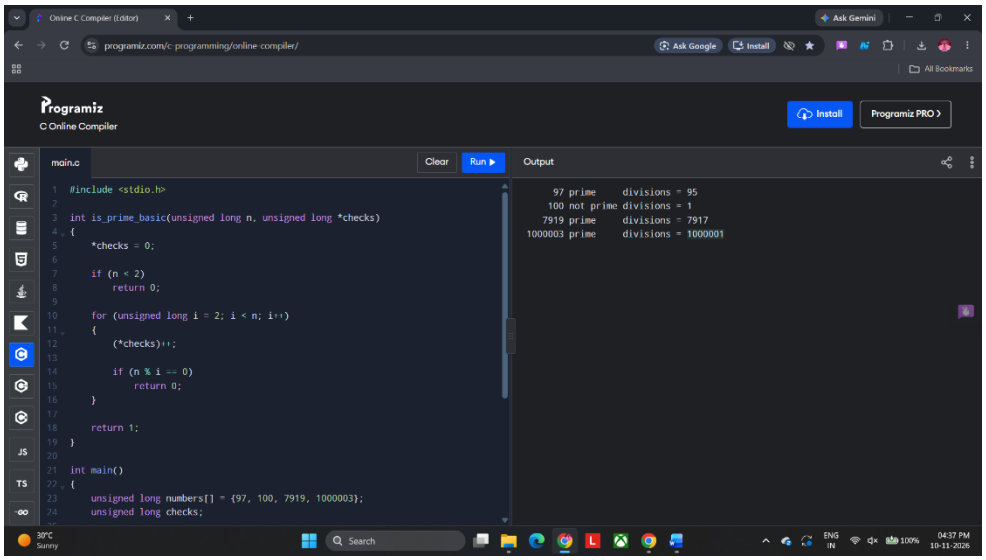
    for (int i = 0; i < 4; i++)
    {
        int result = is_prime_basic(numbers[i], &checks);

        printf("%8lu %-9s divisions = %lu\n",
               numbers[i],
               result ? "prime" : "not prime",
               checks);
    }

    return 0;
}
```

Expected Output of the Basic Program

```
      97 prime     divisions = 95
     100 not prime divisions = 1
    7919 prime     divisions = 7917
 1000003 prime     divisions = 1000001
```



5.3 Improved Prime-Checking Program

```
#include <stdio.h>

unsigned long checks = 0;

int is_prime(unsigned long n)
{
    checks = 0;

    if (n < 2)
        return 0;

    for (unsigned long i = 2; i <= n / i; i++)
    {
        checks++;

        if (n % i == 0)
            return 0;
    }

    return 1;
}

int main()
{
    unsigned long tickets[] = {97, 100, 7919, 1000003};

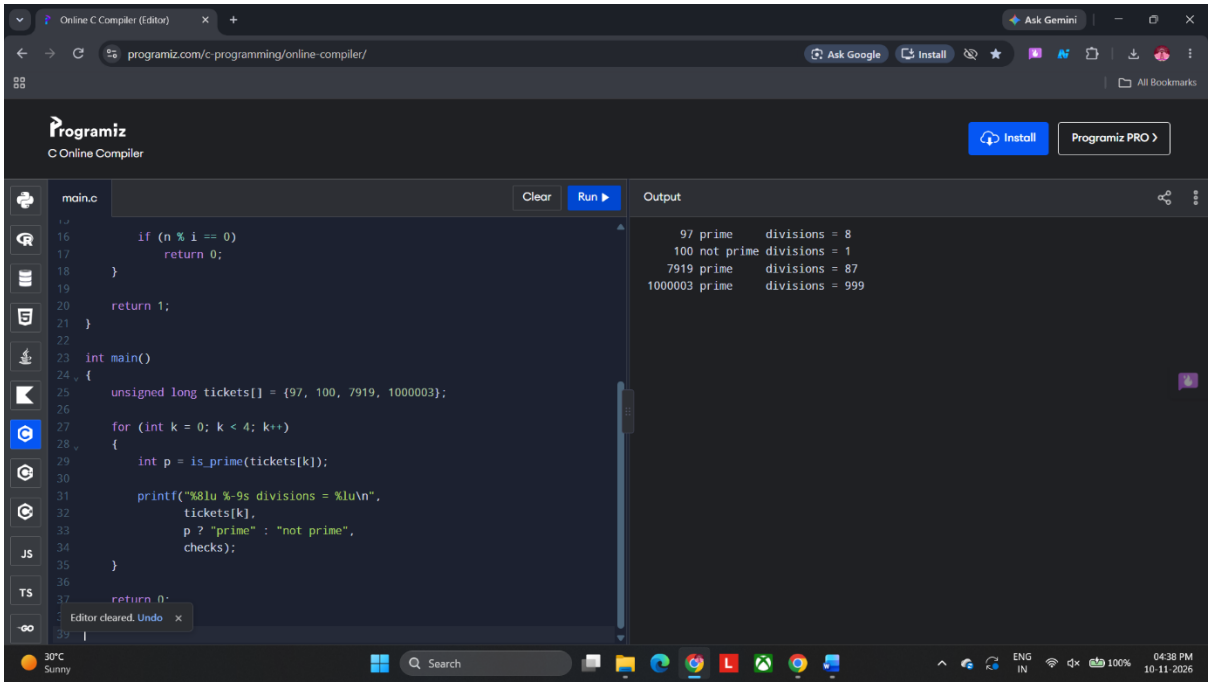
    for (int k = 0; k < 4; k++)
    {
        int p = is_prime(tickets[k]);

        printf("%8lu %-9s divisions = %lu\n",
               tickets[k],
               p ? "prime" : "not prime",
               checks);
    }

    return 0;
}
```

Expected Output of the Improved Program

```
      97 prime     divisions = 8
     100 not prime divisions = 1
    7919 prime     divisions = 87
 1000003 prime     divisions = 999
```



5.5 Division Count Comparison

Number	Basic Loop (2 to n−1)	Improved Loop (up to √n)
97	95	8
100	1	1
7919	7917	87
1000003	1000001	999

For composite numbers, the loop stops when it finds a divisor. This is why the number 100 requires only one division in both versions.

5.6 Improved Loop Using While

```
int is_prime_while(unsigned long n, unsigned long *checks)
{
    *checks = 0;

    if (n < 2)
        return 0;

    unsigned long i = 2;

    while (i <= n / i)
    {
        (*checks)++;

        if (n % i == 0)
            return 0;

        i++;
    }

    return 1;
```

```
}
```

5.7 Improved Loop Using Do-While

```
int is_prime_do(unsigned long n, unsigned long *checks)
{
    *checks = 0;

    if (n < 2)
        return 0;

    if (n == 2)
        return 1;

    if (n % 2 == 0)
    {
        (*checks)++;
        return 0;
    }

    unsigned long i = 3;

    do
    {
        (*checks)++;

        if (n % i == 0)
            return 0;

        i += 2;

    } while (i <= n / i);

    return 1;
}
```

The do-while loop executes its body at least once. Therefore, special handling is required for values such as 2 and other small numbers.

5.8 Changing the Loop Condition

If the improved loop condition is changed from:

```
i <= n / i
```

to:

```
i < n
```

the program checks more divisors than necessary. For the four selected inputs, the counts become the same as the basic loop:

Number	Divisions with `i < n`
97	95
100	1

7919	7917
1000003	1000001

5.9 Alternative Concept – Sieve of Eratosthenes

If the college needs to verify every ticket number up to a fixed limit, the Sieve of Eratosthenes can be used.

The method initially marks all numbers as prime. It then removes multiples of each prime number. After the sieve is completed, checking whether a number is prime takes constant time.

Method	Time Complexity	Space Complexity
Basic trial division	O(n) per number	O(1)
Improved trial division	O(√n) per number	O(1)
Sieve of Eratosthenes	O(N log log N) preprocessing	O(N)

Conclusion: The improved square-root method performs fewer divisions than the basic method. The Sieve is more useful when many prime queries must be answered within a fixed range.

Q6 – LOOPS THAT MUST TERMINATE: LOAN REPAYMENT SCHEDULE

6.1 Problem Analysis

A self-help group provides loans with a monthly interest rate. Each month, interest is added to the outstanding balance and the member pays a fixed repayment amount.

The program must continue until the loan is fully repaid. It should display the monthly opening balance, interest, payment and closing balance.

The selected example is:

Parameter	Value
Principal	₹10,000
Monthly Interest Rate	2%
Monthly Repayment	₹2,000
Maximum Months	360

A while loop is suitable because the number of months is not known in advance.

6.2 Hand Calculation

Monthly interest is calculated as:

Interest = Opening Balance × 2 / 100

Closing Balance = Opening Balance + Interest − Payment

The final payment must be reduced if the remaining balance is less than the fixed repayment amount.

Month	Opening Balance (₹)	Interest (₹)	Payment (₹)	Closing Balance (₹)
1	10000.00	200.00	2000.00	8200.00
2	8200.00	164.00	2000.00	6364.00
3	6364.00	127.28	2000.00	4491.28
4	4491.28	89.83	2000.00	2581.11
5	2581.11	51.62	2000.00	632.73
6	632.73	12.65	645.38	0.00

Total interest paid = ₹645.38
Total repayment = ₹10,645.38
Number of months = 6

6.3 C Program

```
#include <stdio.h>
#include <math.h>

int main()
{
    double balance = 10000.00;
    double rate = 0.02;
    double repayment = 2000.00;
    double total_interest = 0.0;
    double interest, payment;

    int month = 0;
    int max_months = 360;

    printf("Month\tOpening\t\tInterest\tPayment\t\tClosing\n");

    while (balance > 0.005)
    {
        if (month >= max_months)
        {
            printf("Maximum month limit reached.\n");
            break;
        }

        month++;

        interest = round(balance * rate * 100.0) / 100.0;
        total_interest += interest;
```

```
        double amount_due = balance + interest;

        if (repayment <= interest)
        {
            printf("Repayment is insufficient to reduce the loan.\n");
            break;
        }

        if (amount_due < repayment)
            payment = amount_due;
        else
            payment = repayment;

        balance = round((amount_due - payment) * 100.0) / 100.0;

        if (balance < 0.005)
            balance = 0.0;

        printf("%d\t%.2f\t\t%.2f\t\t%.2f\t\t%.2f\n",
               month, amount_due - interest, interest,
               payment, balance);
    }

    if (balance == 0.0)
    {
        printf("\nLoan completed in %d months.\n", month);
        printf("Total Interest: Rs. %.2f\n", total_interest);
        printf("Total Repayment: Rs. %.2f\n",
               10000.00 + total_interest);
    }

    return 0;
}
```

Compilation command:

```
gcc q6.c -o q6 -lm
```

The `-lm` option links the mathematics library used by the `round()` function.

6.4 Loop Termination

The condition `balance != 0` should be avoided because floating-point calculations may not produce an exact zero.

A small tolerance such as `balance > 0.005` can be used when working with amounts rounded to two decimal places.

If the repayment amount is less than or equal to the monthly interest, the outstanding balance will not decrease. The program must stop and display an appropriate message.

6.5 Growth Investigation

The following values show how the repayment amount affects the number of months. These are calculated using the same monthly interest and rounding approach.

Monthly Repayment	Approximate Months
₹2,000	6
₹500	26
₹250	82
₹210	154
₹205	188
₹201	268
₹200	No progress
₹199	No progress

As the repayment amount approaches the monthly interest, the number of months increases significantly.

6.6 Alternative Concept – Direct Loan Formula

The number of months can also be estimated using the formula:

[
n = \frac{-\log(1-rP/E)}{\log(1+r)}
]

Where:

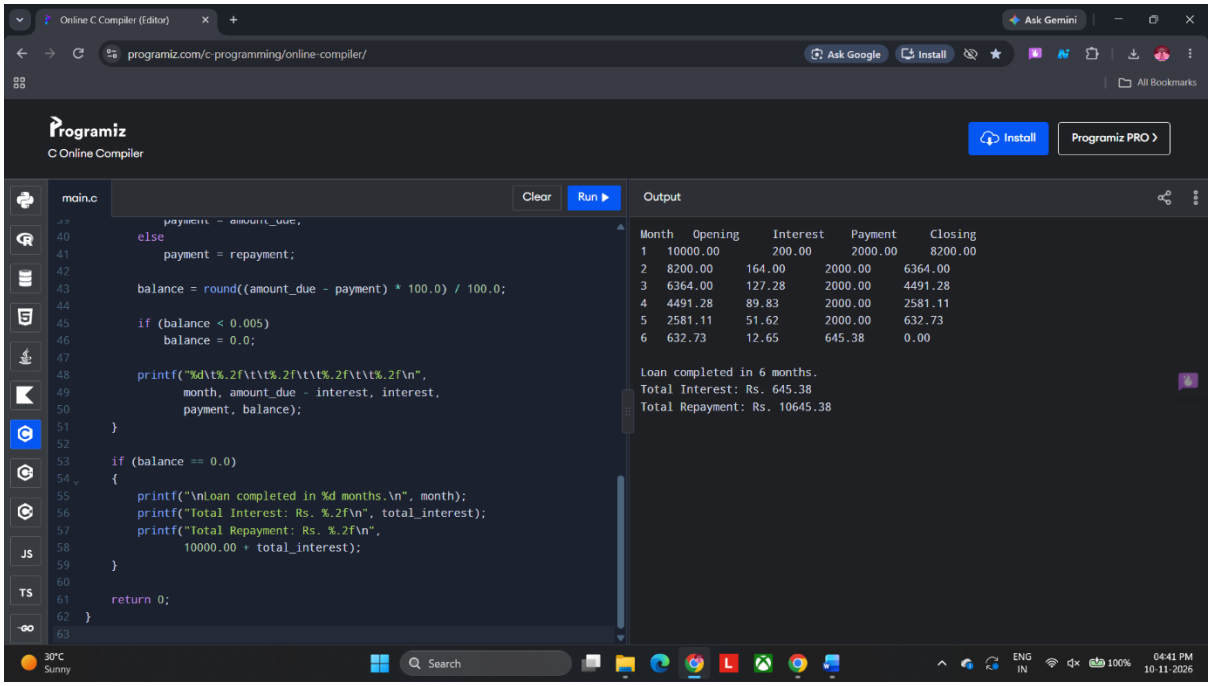
- P = Principal amount
- r = Monthly interest rate
- E = Fixed monthly repayment
- n = Number of months

This formula assumes a fixed interest rate and payment schedule. The result may differ slightly from the monthly simulation because the program rounds interest and adjusts the final payment.

Factor	Monthly Simulation	Direct Formula
Time Complexity	O(n)	O(1)
Extra Space	O(1)	O(1)
Monthly Schedule	Available	Must be calculated separately
Rounding	Can be applied every month	May differ from rounded schedule
Suitability	Detailed repayment schedule	Quick estimate

Conclusion: A while loop is suitable for generating the month-by-month repayment schedule. A maximum iteration limit and an insufficient-payment check help prevent an endless loop.

6.7 Expected Output



Q7 – DIAGNOSE, REDESIGN AND VALIDATE

7.1 Selected Program

The prime-number checking program from Q5 is selected for redesign.

The original improved program checks every divisor from 2 up to the square root of the number. Although it is better than checking every number up to n − 1, it still performs unnecessary checks for even divisors after checking 2.

7.2 A. Diagnose the Bottleneck

The main repeated operation in the original program is the modulus operation:

```
n % i
```

The program checks 2, 3, 4, 5 and continues up to the square root of n.

For prime numbers, the loop performs all these checks because no divisor is found.

Input Number	Original Division Checks
97	8
100	1
7919	87
1000003	999

The number of checks increases as the input number becomes larger.

7.3 B. Design Objective

The objective is to reduce the number of division operations while maintaining the same prime-number results.

The redesigned program will:

1. Handle numbers less than 2.
2. Handle the number 2 separately.
3. Reject other even numbers.
4. Check only odd divisors starting from 3.
5. Stop when the divisor exceeds the square root of the number.

7.4 C. Revised Solution

The redesigned program skips even divisors because every even number greater than 2 is not prime.

Pseudocode:

```
START

Read n

IF n < 2
    Return Not Prime

IF n = 2
    Return Prime

IF n is divisible by 2
    Return Not Prime

Set i = 3

WHILE i <= n / i
    Increase division count

    IF n is divisible by i
        Return Not Prime

    i = i + 2

Return Prime
```

```
STOP
```

7.5 Redesigned C Program

```
#include <stdio.h>

int is_prime_optimized(unsigned long n, unsigned long *checks)
{
    *checks = 0;

    if (n < 2)
        return 0;

    if (n == 2)
        return 1;

    (*checks)++;

    if (n % 2 == 0)
        return 0;

    for (unsigned long i = 3; i <= n / i; i += 2)
    {
        (*checks)++;

        if (n % i == 0)
            return 0;
    }

    return 1;
}

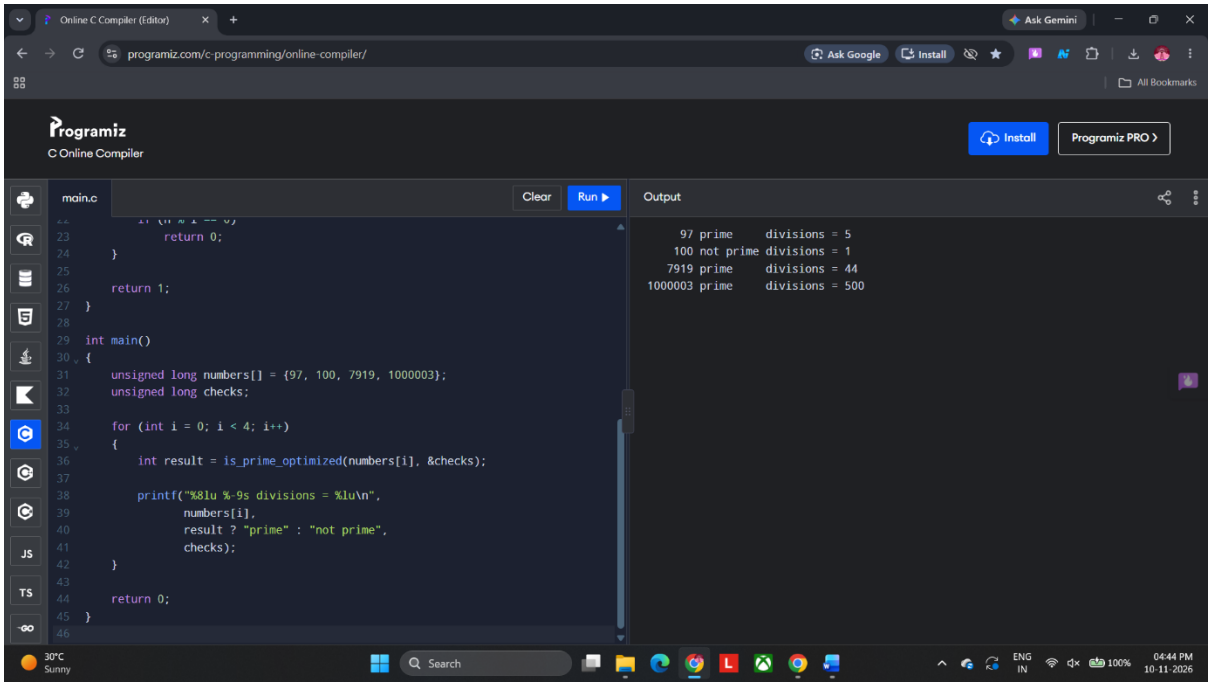
int main()
{
    unsigned long numbers[] = {97, 100, 7919, 1000003};
    unsigned long checks;

    for (int i = 0; i < 4; i++)
    {
        int result = is_prime_optimized(numbers[i], &checks);

        printf("%8lu %-9s divisions = %lu\n",
               numbers[i],
               result ? "prime" : "not prime",
               checks);
    }

    return 0;
}
```

7.6 Expected Output



7.7 D. Prediction Before Implementation

The redesigned program is expected to reduce the number of division operations because it skips even divisors after checking 2.

The improvement will be measured by comparing the number of divisions performed by the original and redesigned programs using the same inputs.

7.8 E. Original and Redesigned Comparison

Number	Original Checks	Redesigned Checks	Reduction
97	8	5	3
100	1	1	0
7919	87	44	43
1000003	999	500	499

For the large prime number 1,000,003, the division count is reduced from 999 to 500.

Percentage reduction:

[
\frac{999-500}{999}\times100 \approx 49.95%
]

7.9 F. Final Design Evaluation

Factor	Original Improved Program	Redesigned Program
Divisor checking	Checks even and odd divisors	Checks 2 and then odd divisors
Time Complexity	O(√n)	O(√n)
Space Complexity	O(1)	O(1)
Division count for 1,000,003	999	500
Implementation	Simple	Slightly more conditions

The redesigned program achieves almost a 50% reduction in division checks for the selected large prime number. Both programs have the same Big-O time complexity, but the redesigned version performs fewer operations in practice.

The original version may be easier for beginners to understand because its loop is simpler. However, the redesigned version is more efficient for larger prime numbers.

Conclusion: The redesign successfully reduces unnecessary even-divisor checks while preserving the correctness of prime-number detection. It improves the practical performance without requiring additional memory.

OVERALL CONCLUSION

This assignment provided practical experience with operators, decision-making statements and looping constructs in C programming.

The pharmacy billing problem demonstrated the importance of operator precedence, associativity, integer division and readable expressions. The railway concession problem explained short-circuit evaluation and how condition ordering can reduce unnecessary function calls.

The smart-home controller demonstrated the use of bitwise operators to store multiple device states efficiently. The grade-classification problem compared else-if ladders, switch statements and lookup tables.

The prime-number problem showed how loop conditions affect the number of operations performed. The loan repayment program demonstrated loop termination, floating-point handling and safety limits.

Finally, the redesign of the prime-number program showed how analysing operation counts can identify unnecessary work and improve program efficiency.

Overall, the assignment helped develop skills in writing, testing, analysing and improving C programs.